Design Document

1.Introduction

Machine learning models often achieve high predictive performance during development but require additional engineering components before they can be deployed in a production environment. A production-ready machine learning system must not only train accurate models but also support reliable inference, data ingestion, monitoring, retraining decisions, and reproducible workflows.

This project implements a miniature production machine learning system for customer churn prediction using the IBM Telco Customer Churn dataset. The implementation extends beyond traditional model development by incorporating modular data preprocessing, feature engineering, model evaluation, REST API deployment using FastAPI, batch data ingestion, data quality monitoring, feature drift detection, retraining decision logic, and latency measurement.

The primary objective of the project is to demonstrate how machine learning engineering concepts can be applied to build a maintainable and reproducible end-to-end system rather than focusing solely on maximizing predictive accuracy. Throughout the project, emphasis is placed on modular software design, reproducible pipelines, and production-oriented decision making, closely reflecting the complete machine learning lifecycle.

2. Problem Definition and Success Metrics

Customer churn prediction is a binary classification problem in which the objective is to determine whether a customer is likely to discontinue using a company's services. Early identification of customers with a high probability of churn enables organizations to implement targeted retention strategies, reducing customer acquisition costs and improving long-term revenue.

The IBM Telco Customer Churn dataset was selected because it contains customer demographic information, subscribed services, contract details, and billing history that are representative of real-world customer retention problems. The target variable is **Churn**, where a value of **1** indicates that a customer has churned and **0** indicates that the customer has remained with the company.

As this is a binary classification task, several evaluation metrics were used to assess model performance. Accuracy provides an overall measure of correct predictions, while Precision and Recall evaluate the model's ability to correctly identify customers at risk of churn. The F1-score balances Precision and Recall, making it particularly useful when considering the trade-off between false positives and false negatives. In addition, the Receiver Operating Characteristic – Area Under the Curve (ROC-AUC) was selected as the primary metric for model comparison because it measures the model's ability to

distinguish between churn and non-churn customers across multiple decision thresholds.

Rather than automatically selecting the model with the highest evaluation metric, the project implements simple promotion guardrails that reflect production deployment practices. A candidate model is promoted only if it achieves a minimum acceptable ROC-AUC while also maintaining performance comparable to the currently deployed baseline model.

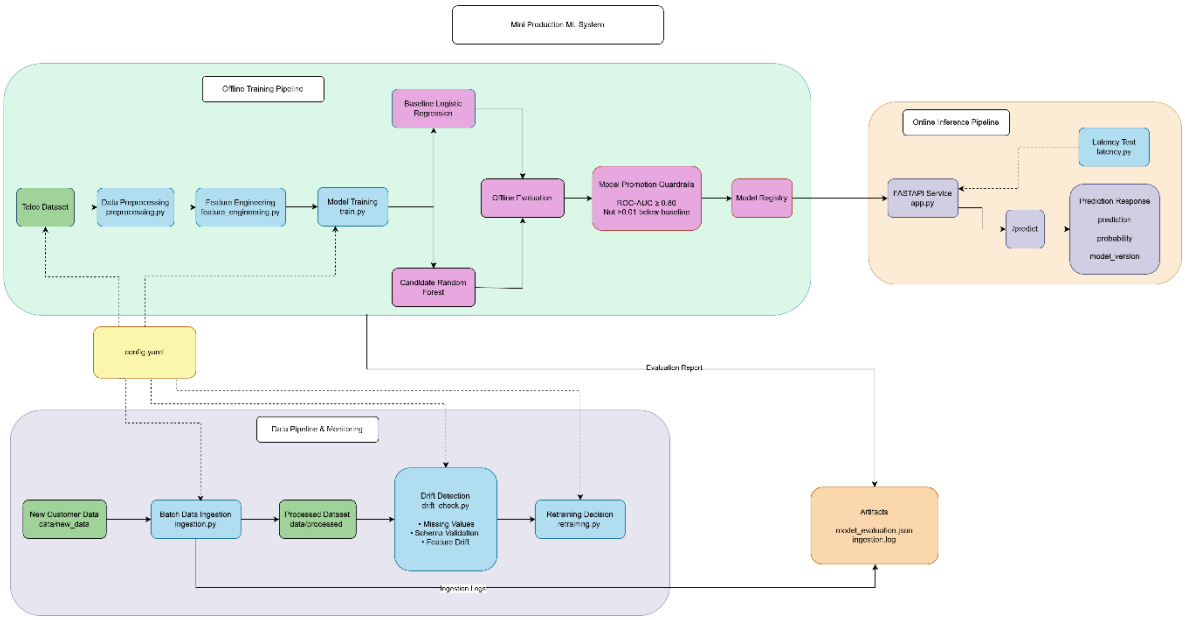

Figure 1 Overall architecture of the Telco Churn Prediction System showing the offline training pipeline, online inference service, and monitoring workflow.

3. Dataset Description & Data Preprocessing

Dataset Description

The project uses the **IBM Telco Customer Churn** dataset, which is widely used for binary classification problems involving customer retention. The dataset contains customer demographic information, account details, subscribed services, billing information, and contract attributes. Each record represents a single customer, with the target variable **Churn** indicating whether the customer has left the service.

The dataset initially contained **7,043 customer records** and **21 attributes**. After preprocessing, **11 records** containing invalid or missing values were removed, resulting in a final dataset of **7,032 records** used for model training and evaluation. The dataset includes both numerical and categorical features, requiring different preprocessing techniques before model training.

Data Preprocessing

To ensure consistency between model training and inference, all preprocessing operations were implemented within a shared **preprocess_data()** function in preprocessing.py. Using a common preprocessing module prevents training-serving

skew by ensuring that identical transformations are applied during both offline model training and online prediction.

The preprocessing pipeline consisted of the following steps:

- Removal of customer identifier (customerID), as it does not contribute predictive information.
- Conversion of the **TotalCharges** column from string to numeric format.
- Removal of records containing missing or invalid values.
- Conversion of the target variable **Churn** into a binary numerical representation, where **Yes = 1** and **No = 0**.
- Validation of the cleaned dataset before passing it to the feature engineering stage.

Separating preprocessing into an independent module improves code reusability, simplifies maintenance, and ensures that any future modifications to the preprocessing logic are automatically reflected in both the training pipeline and the inference service.

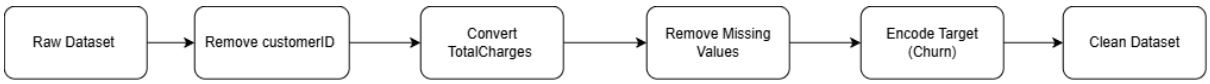

Figure 2 Data preprocessing workflow applied before feature engineering.

4. Feature Engineering

Feature engineering was performed after data preprocessing to create additional variables that capture customer behaviour more effectively than the original dataset alone. All engineered features were implemented within a dedicated **create_features()** function in feature_engineering.py, ensuring that the same transformations can be reused during both model training and inference.

Five engineered features were introduced:

Feature	Description
AvgMonthlySpend	Ratio of TotalCharges to tenure, representing the customer's average monthly expenditure.
IsNewCustomer	Binary indicator identifying customers with less than 12 months of tenure.
IsMonthToMonth	Binary indicator identifying customers on month-to-month contracts.

HasFiberOptic	Binary indicator identifying customers subscribed to Fiber Optic internet services.
ServiceCount	Count of subscribed services such as phone, online backup, streaming services, device protection, and technical support.

These engineered features were selected because they represent customer engagement, contract commitment, and spending behaviour, all of which are expected to influence customer churn.

From a production perspective, these features can be divided into **offline** and **online** features. The engineered variables are calculated during offline model training but are also generated dynamically during API inference using the same feature engineering module. This shared implementation ensures feature consistency and prevents discrepancies between the training environment and the deployed prediction service.

By centralizing feature engineering into a reusable module, the project minimizes training-serving skew and improves maintainability, allowing future feature additions to be implemented in a single location without requiring changes across multiple components of the system.

5. Model Training & Offline Evaluation

Model Training Pipeline

The model training process was implemented as a repeatable and modular pipeline within **train.py**. Rather than developing the model inside a notebook, the complete workflow was structured as a sequence of reusable functions that automate data loading, preprocessing, feature engineering, model training, evaluation, and model persistence.

The training pipeline follows the sequence below:

1. Load the raw dataset.
2. Apply the shared preprocessing module.
3. Generate engineered features.
4. Split the dataset into training and testing sets.
5. Build the preprocessing pipeline for numerical and categorical features.
6. Train the baseline and candidate models.
7. Evaluate both models using multiple classification metrics.
8. Apply production promotion guardrails.
9. Save the selected production model.

10. Store evaluation results for future reference.

Structuring the workflow as a repeatable pipeline improves reproducibility and reduces manual intervention during model development. Each stage is isolated into dedicated helper functions, making the pipeline easier to maintain and extend.

Baseline and Candidate Models

Two classification models were evaluated during development.

The **Baseline Model** uses **Logistic Regression**, which is widely adopted as a benchmark for binary classification problems due to its simplicity, interpretability, and computational efficiency. Logistic Regression provides a strong reference point against which more complex models can be compared.

The **Candidate Model** uses a **Random Forest Classifier**, an ensemble learning algorithm capable of modelling non-linear relationships and feature interactions that may not be captured by Logistic Regression. Random Forest was selected as the candidate model to determine whether the increased model complexity resulted in improved predictive performance.

Both models were implemented using identical preprocessing pipelines to ensure that the comparison was fair and that performance differences resulted solely from the learning algorithms rather than differences in feature preparation.

Offline Evaluation

Model performance was evaluated using a hold-out test dataset that was not used during training. Multiple evaluation metrics were considered to provide a balanced assessment of predictive performance.

The following metrics were used:

- **Accuracy** measures the overall proportion of correctly classified customers.
- **Precision** measures the proportion of predicted churn cases that were actually churn events.
- **Recall** measures the model's ability to correctly identify customers who eventually churned.
- **F1-score** balances Precision and Recall into a single metric.
- **ROC-AUC** evaluates the model's ability to distinguish between churn and non-churn customers across multiple decision thresholds and was selected as the primary metric for model comparison.

The evaluation results are summarised below.

Metric	Logistic Regression	Random Forest
Accuracy	**0.7939**	0.7875
Precision	**0.6304**	0.6221
Recall	**0.5428**	0.5107
F1-score	**0.5833**	0.5609
ROC-AUC	**0.8340**	0.8155

Although Random Forest is capable of modelling more complex relationships, Logistic Regression achieved the highest ROC-AUC while remaining simpler and more interpretable. Therefore, it was retained as the production model.

The results indicate that the baseline Logistic Regression model achieved superior performance across all evaluation metrics, including the highest ROC-AUC score. Although the Random Forest model demonstrated competitive performance, it did not provide a meaningful improvement over the simpler baseline model.

Model Promotion Strategy

Rather than automatically deploying the highest-scoring model, the project implements a simplified production-style model promotion strategy using deployment guardrails.

A candidate model is promoted only if it satisfies both of the following conditions:

- Achieves a minimum **ROC-AUC of 0.80**.
- Performs **no more than 0.01 below the baseline ROC-AUC**.

These guardrails simulate production deployment practices by preventing unnecessary model replacement when a new model offers little or no practical improvement.

Using this strategy, the Random Forest model satisfied the minimum acceptable ROC-AUC threshold but failed the second guardrail because its ROC-AUC score was more than 0.01 below the baseline model. Consequently, the Logistic Regression model remained the selected production model and was serialized for deployment through the FastAPI inference service.

Finally, the selected model was saved as a serialized artifact using **Joblib**, while the evaluation metrics were stored as a JSON report within the **artifacts** directory. Persisting these outputs supports reproducibility and provides a record of model performance for future comparison during retraining.

6. Serving & Inference Pattern

Model Serving

After the model selection process, the chosen production model was deployed as a RESTful inference service using **FastAPI**. FastAPI was selected because it provides a lightweight, high-performance framework for exposing machine learning models through HTTP endpoints while automatically generating interactive API documentation.

During application startup, the serialized production model is loaded from the **models** directory using Joblib. Loading the model once during initialization avoids repeated disk access and reduces prediction latency for subsequent requests.

The inference service exposes two API endpoints:

- **GET /** – Returns a simple welcome message confirming that the API is running successfully.
- **POST /predict** – Accepts customer information in JSON format and returns a churn prediction.

Incoming requests are validated using a Pydantic schema before prediction. This ensures that all required fields are present and correctly typed, reducing the likelihood of invalid requests reaching the prediction pipeline.

Before inference, the incoming customer data is converted into a Pandas DataFrame and passed through the shared preprocessing and feature engineering modules. Reusing the same preprocessing functions that were used during model training ensures feature consistency and minimizes training-serving skew.

The API response includes three values:

- **prediction** – Binary churn prediction (0 = No Churn, 1 = Churn).
- **churn_probability** – Predicted probability that the customer will churn.
- **model_version** – Identifier of the deployed production model.

Including the model version improves traceability by allowing predictions to be associated with the exact model that generated them.

Inference Pattern

This project adopts an **online request-response inference pattern**, where predictions are generated immediately when a client submits a request to the API.

This approach was selected because customer churn prediction is commonly used in customer relationship management systems, where predictions may be required during customer interactions, support calls, or account management activities. In these scenarios, users expect predictions within a short response time rather than waiting for scheduled batch processing.

The request-response architecture also simplifies deployment by separating model training from model inference. Models are trained offline and only the selected production model is deployed to the inference service.

API Performance Measurement

To evaluate inference performance, a dedicated latency testing script (**latency_test.py**) was developed. The script repeatedly sends prediction requests to the FastAPI endpoint and measures the response time for each request.

Rather than reporting only the average latency, several performance statistics are calculated:

- Number of successful requests.
- Number of failed requests.
- Average latency.
- Minimum latency.
- Maximum latency.
- 95th percentile (P95) latency.

The inclusion of the P95 latency metric provides a more representative measure of user experience, as it reflects the response time experienced by the majority of requests while excluding occasional outliers. This metric is widely used in production systems to evaluate API responsiveness.

The latency measurements confirmed that the deployed inference service is capable of responding consistently under repeated requests while maintaining low response times suitable for online inference.

Design Decisions

Several implementation decisions were made to improve maintainability and reliability:

- The deployed model is loaded only once during application startup, avoiding repeated model deserialization.
- Shared preprocessing and feature engineering modules are reused during both training and inference to ensure feature consistency.
- Request validation is performed using Pydantic models before predictions are generated.
- The inference service returns both the predicted class and prediction probability, allowing downstream applications to make threshold-based decisions if required.
- Model version information is included in every prediction response to improve traceability and simplify future model upgrades.

These design decisions collectively improve the robustness of the inference service while maintaining a clear separation between model training and model deployment.

The inference service was designed to run on standard CPU hardware, avoiding the additional operational cost of GPU-based deployment while maintaining low response latency for this workload.

7. Data Pipeline & Retraining Strategy

Data Pipeline

A production machine learning system must be capable of incorporating newly available data into the training process. To simulate this behaviour, a simple batch data ingestion pipeline was implemented using **ingestion.py**.

The ingestion process reads newly available customer records from the **data/new_data** directory and combines them with the existing training dataset. The merged dataset is then written to the **processed** data directory, providing an updated dataset that can be used during future model retraining.

In addition to updating the processed dataset, the ingestion pipeline records metadata about each ingestion event, including the timestamp, number of rows added, and the total number of records after ingestion. This information is written to an ingestion log stored within the **artifacts** directory, providing a simple audit trail of data updates.

Although the implementation uses CSV files for simplicity, the same design could be extended to production data sources such as cloud storage, relational databases, or streaming platforms without changing the overall pipeline architecture.

Retraining Strategy

Training a model only once is rarely sufficient in production, as customer behaviour and business conditions evolve over time. To address this challenge, the project implements a simplified retraining decision module in **retraining.py**.

Rather than automatically retraining whenever new data becomes available, the system evaluates several conditions before recommending retraining. This mirrors production environments where retraining is typically triggered only when there is evidence that the current model may no longer perform adequately.

The implemented retraining strategy considers the following signals:

- Significant feature drift detected in newly ingested data.
- Accumulation of sufficient new customer records.
- Reduction in model performance measured through future evaluation metrics.

If one or more of these conditions are satisfied, the system recommends retraining the model using the updated dataset. Separating the retraining decision from the training

pipeline improves maintainability and allows the retraining logic to evolve independently of the model development process.

Although the current implementation reports whether retraining should occur, it does not automatically retrain the model. This design reflects common production practices where retraining decisions are often reviewed or orchestrated through scheduled workflows before deployment.

8. Monitoring Plan

Monitoring Strategy

Reliable machine learning systems require continuous monitoring after deployment to ensure that both the infrastructure and the underlying data remain healthy. This project implements a lightweight monitoring module in **drift_check.py** to simulate production monitoring practices.

The monitoring process focuses on three categories of metrics:

Infrastructure Monitoring

The deployed FastAPI service is monitored through latency measurements obtained using the dedicated latency testing script. Key performance indicators include:

- Average response latency
- Maximum response latency
- 95th percentile (P95) latency
- Number of successful requests
- Number of failed requests

These metrics provide visibility into API responsiveness and help identify potential performance degradation.

Data Quality Monitoring

Before new data is incorporated into the training pipeline, several validation checks are performed:

- Detection of missing values.
- Verification that the incoming dataset matches the expected schema.
- Basic validation of feature consistency.

These checks help prevent invalid or incomplete data from entering the machine learning pipeline.

Feature Drift Monitoring

Customer behaviour may change over time, causing the statistical properties of input features to differ from those observed during model training. To detect these changes, the monitoring module compares summary statistics of selected numerical features between the original training dataset and newly ingested data.

If the observed difference exceeds a predefined threshold, the system reports potential feature drift. While this approach is intentionally lightweight, it demonstrates the concept of monitoring data distributions before model performance begins to deteriorate.

Incident Scenario

One possible production incident is a change in the format or quality of incoming customer data. For example, if new records contain unexpected missing values or incompatible data types, the preprocessing pipeline may fail or produce inconsistent features.

Within this project, such issues are identified through the schema validation and missing value checks implemented in the monitoring module. If these checks fail, the issue can be investigated before the data is incorporated into the training dataset, preventing downstream model failures.

Similarly, if feature drift exceeds the defined threshold or future model evaluation indicates a significant reduction in predictive performance, the retraining module recommends rebuilding the production model using updated customer data.

This combination of monitoring and retraining logic helps ensure that the deployed model remains reliable as new data becomes available.

9. Trade-offs, Limitations & Future Work

Design Trade-offs

The primary objective of this project was to demonstrate the complete lifecycle of a production-oriented machine learning system while maintaining a clear and modular implementation. Several design decisions were made to balance simplicity, reproducibility, and maintainability.

The system uses **Logistic Regression** as the deployed production model, despite evaluating a more complex Random Forest classifier. Although Random Forest is capable of capturing non-linear relationships, the evaluation results showed that it did not outperform the baseline model. Retaining the simpler model improves interpretability, reduces computational overhead, and aligns with the project's promotion guardrails.

Another design decision was the use of **CSV files** for data storage and ingestion rather than a database or cloud-based storage system. While this approach is sufficient for demonstrating the data pipeline within the scope of this project, production systems would typically ingest data from distributed storage systems, relational databases, or streaming platforms.

The current implementation is intentionally lightweight and runs on a single FastAPI instance with CSV-based storage, resulting in minimal infrastructure cost. In a production deployment, additional costs would arise from scalable cloud compute resources, managed databases, monitoring platforms, and orchestration services required to support larger workloads.

Similarly, the project implements a lightweight monitoring framework using statistical checks and predefined thresholds rather than advanced monitoring platforms. This design was chosen to clearly demonstrate monitoring concepts without introducing unnecessary infrastructure complexity.

Finally, model deployment was implemented using a single FastAPI application running locally. While this is appropriate for demonstrating online inference, production deployments would normally include containerization, orchestration, load balancing, authentication, and automated deployment pipelines.

Limitations

Although the implemented system demonstrates the key components of a production machine learning workflow, several limitations remain.

The project evaluates only two supervised learning algorithms. Exploring additional models, such as Gradient Boosting or XGBoost, may further improve predictive performance.

The monitoring module performs basic statistical comparisons to identify potential feature drift. More sophisticated drift detection techniques, including statistical hypothesis testing or dedicated monitoring frameworks, could provide earlier and more reliable detection of data distribution changes.

Retraining decisions are currently recommendation-based and require manual execution of the training pipeline. In a production environment, retraining would typically be automated using workflow orchestration tools or scheduled jobs.

The API currently processes one customer record per request. Supporting batch inference could improve throughput for large-scale prediction workloads.

Finally, the system assumes that incoming data follows the expected schema. While basic validation is implemented, production systems often include more comprehensive schema enforcement, versioning, and automated validation pipelines.

Cost Considerations

The current implementation is intentionally lightweight and designed for educational purposes. The system runs as a single FastAPI application using CSV files for data storage and a locally deployed machine learning model, resulting in minimal infrastructure costs during development and testing.

In a production environment, operational costs would increase as the system scales. Additional expenses may include cloud compute resources for serving prediction

requests, managed databases or object storage for datasets, monitoring and logging services, and workflow orchestration tools for automated retraining. Infrastructure costs would also depend on factors such as request volume, model complexity, and availability requirements.

To balance performance and cost, the deployed model was kept relatively lightweight, enabling low-latency predictions without requiring specialised hardware. This makes the current solution suitable for small to medium-scale workloads while providing a foundation that can be extended for larger deployments.

Future Work

Several enhancements could further improve the production readiness of the system.

Future work may include:

- Containerizing the application using Docker to simplify deployment across different environments.
- Implementing Continuous Integration and Continuous Deployment (CI/CD) pipelines to automate testing and deployment.
- Integrating a model registry to manage multiple model versions and deployment history.
- Replacing CSV-based data storage with a relational database or cloud object storage.
- Introducing automated retraining schedules using workflow orchestration tools such as Apache Airflow.
- Expanding the monitoring framework with statistical drift detection techniques and real-time dashboards.
- Supporting batch prediction endpoints alongside the existing online inference API.
- Increasing test coverage through additional unit, integration, and end-to-end tests.

These enhancements would further align the project with enterprise-scale machine learning systems while preserving the modular architecture established during this implementation.

10. Conclusion

This project demonstrates the implementation of a miniature production machine learning system for customer churn prediction using the IBM Telco Customer Churn dataset. Rather than focusing solely on model development, the project incorporates

several engineering components required to support the complete machine learning lifecycle.

The final implementation includes modular data preprocessing, reusable feature engineering, repeatable model training, offline evaluation, production-style model promotion guardrails, REST API deployment through FastAPI, batch data ingestion, data quality monitoring, feature drift detection, retraining decision logic, and API latency measurement. These components were designed as independent modules, improving maintainability, reproducibility, and consistency across both training and inference workflows.

Although the system has been intentionally simplified for educational purposes, it reflects many of the architectural principles used in real-world machine learning deployments. The project demonstrates how software engineering practices can be combined with machine learning techniques to build systems that are not only accurate but also maintainable, scalable, and suitable for production environments.